4-bit Array Multiplier from RTL to GDSII with Open-Source Tools

Team member	Roll number
Akshat Agrawal	2026203014
Simon Shibu	2026203003

Item	Detail
Repository	github.com/Akshat181102/mult4_array
Technology	SkyWater SKY130 (130 nm), standard cell library sky130_fd_sc_hd, typical corner tt_025C_1v80

1. Introduction

This report documents the design of a 4-bit × 4-bit unsigned array multiplier and its implementation from RTL to GDSII on the SkyWater SKY130 130 nm process with open-source tools. The multiplier forms 16 partial-product bits with AND gates and adds them in three rows of 4-bit ripple-carry adders built from 12 full adders. Its inputs and its 8-bit output are registered, giving 16 flip-flops, a latency of two clock cycles and a throughput of one product per clock cycle.

Section 2 covers RTL design and verification, Section 3 logic synthesis with Yosys, Section 4 static timing analysis with OpenSTA and Section 5 physical design with LibreLane. Section 6 answers the assignment questions. Table 1 lists the tools used.

Table 1. Tools and versions.

Tool	Version	Used for
IIC-OSIC-TOOLS	hpretl/iic-osic-tools:latest (sha256:306585de35d1…)	Container with all tools and the SKY130 PDK
Icarus Verilog	14.0 (devel)	RTL and gate-level simulation
GTKWave	4.0.0-prealpha (f6d60e0)	Waveform viewing
Yosys	0.70	Logic synthesis
OpenSTA	3.1.0	Static timing analysis
LibreLane	3.1.0.dev3	RTL-to-GDSII flow (OpenROAD, Magic, KLayout, Netgen)
KLayout	0.30.12	Layout viewing

2. Part A: RTL Design and Verification

2.1 Design

`full_adder.v` implements a 1-bit full adder with two continuous assignments: `sum = a ^ b ^ cin` and `cout = (a & b) | (a & cin) | (b & cin)`.

`mult_array.v` registers the inputs `a` and `b` into `a_r` and `b_r` on every rising clock edge and builds the array from these registered values. The 16 partial-product bits pp_i[j] = a_r[j] AND b_r[i] are formed with AND operations. Row 0 needs no adders: pp0[0] is product bit p[0], and {0, pp0[3], pp0[2], pp0[1]} is passed down to row 1. Rows 1 to 3 are 4-bit ripple-carry adders of four `full_adder` instances each (12 in total, created with `generate` loops), wired as in the connection table of the assignment, with the carry-in of every row tied to 0. The 8-bit result {c3[4], s3[3], s3[2], s3[1], s3[0], s2[0], s1[0], pp0[0]} is registered into `p`.

The design therefore contains 16 flip-flops (4 + 4 + 8), all cleared by the asynchronous active-low reset `rst_n`. Its latency is two clock cycles and its throughput is one product per clock cycle. The `*` operator is not used in `mult_array.v`.

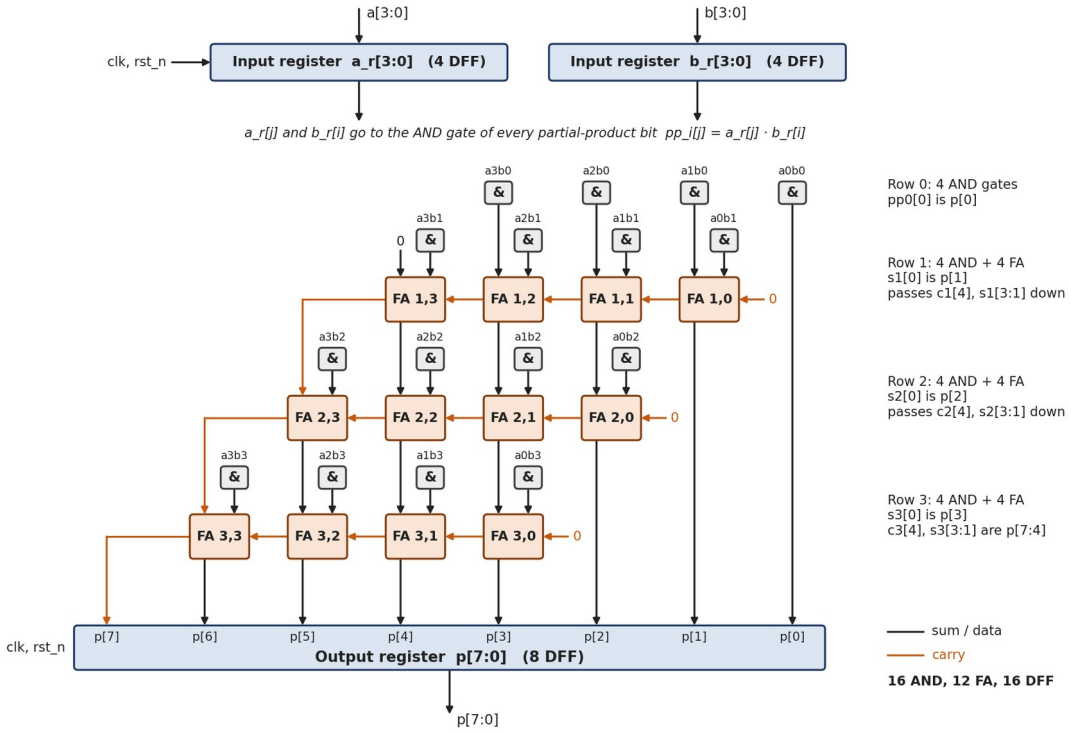


Figure 1. Block diagram of the registered 4-bit array multiplier: 16 AND gates, 12 full adders, input registers and output register. Columns are product-bit weights, so each sum drops straight into the row below and each carry moves one column to the left.

2.2 Verification

`tb_mult.v` generates a 10 ns clock, holds `rst_n` low for the first two clock cycles and then applies all 256 combinations of `a` and `b`, one every two cycles. Inputs change on falling clock edges, half a cycle away from the rising edges at which the registers sample them. For each vector the testbench waits two rising edges for the latency, then compares `p` with `a * b` using `!==`, so that unknown (X) values also count as errors. It prints a line for every mismatch and one summary line at the end, and writes the waveform file `mult.vcd`. Because it uses only the ports of `mult_array`, the same testbench also runs on the synthesized netlist in Part B.

To confirm that the testbench detects errors, the sum output of `full_adder.v` was temporarily changed from `a ^ b ^ cin` to `a ^ b`, which drops the carry-in. The testbench then reported 95 mismatching vectors out of 256 (Table 2). The change was reverted and the design passed again.

Table 2. Testbench results.

Test	Summary line printed by tb_mult
RTL simulation, all 256 input combinations	`PASS: 256 vectors checked, 0 errors`
Error test (sum changed to `a ^ b`)	`FAIL: 256 vectors checked, 95 errors`

2.3 Worked example

The example operands come from Akshat Agrawal's roll number 2026203014: A = last digit + 5 = 4 + 5 = 9 = 1001 (binary) and B = 7 = 0111 (binary). Their product is 9 × 7 = 63 = 0011 1111 (binary); Figure 2 shows the multiplication worked by hand in the partial-product layout.

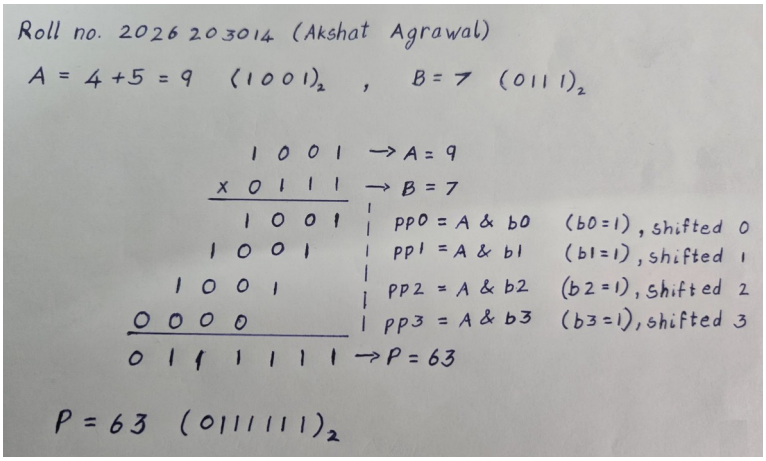


Figure 2. Hand-worked multiplication A × B = 9 × 7 in the partial-product layout of Section 2 of the assignment.

Figure 3 shows the same multiplication in the RTL simulation. The inputs a = 9 and b = 7 are applied at 3040 ns, on a falling clock edge. The input registers capture them at the next rising edge, 3045 ns (marker "edge1: a, b capture"), and the output register captures the product p = 63 at the following rising edge, 3055 ns (marker "edge2: p = 63"). Until then p still shows 54, the product of the previous vector 9 × 6. The latency is therefore two clock cycles, 20 ns at the 10 ns test clock.

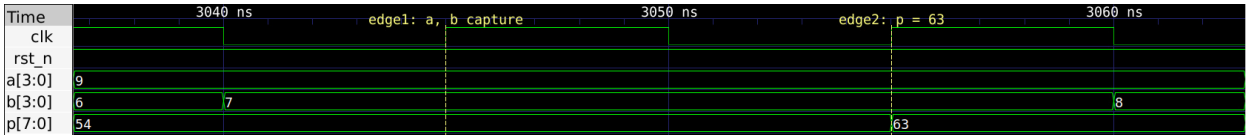


Figure 3. RTL simulation waveform for A = 9, B = 7, with the two-cycle latency marked.

3. Part B: Logic Synthesis with Yosys

`synth/synth.tcl` runs the seven Yosys steps of the assignment: `read_verilog`, `synth -top mult_array -flatten`, `dfflibmap` and `abc` with the typical-corner library `sky130_fd_sc_hd__tt_025C_1v80.lib`, `opt_clean -purge`, `stat -liberty` and `write_verilog -noattr`. Table 3 summarises the resulting report `synth/mult_array_stat.rpt`; the full list of cell types is in Appendix A.

The gate-level netlist `synth/mult_array_netlist.v` was simulated with the same testbench and the SKY130 cell models (`-DFUNCTIONAL -DUNIT_DELAY=#1`) and also passed all 256 vectors, so synthesis did not change the behaviour. The testbench needed no change for this, because it compares `p` half a clock cycle (5 ns) after the clock edge, well after the 1 ns delay that the cell models give the flip-flop outputs.

Table 3. Synthesis results (sky130_fd_sc_hd, tt_025C_1v80).

Metric	Value
Total number of cells	61
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`)
Total cell area (µm²)	756.976
Flip-flop area (µm²)	400.384 (16 × 25.024)
Flip-flop area (% of total)	52.89 %
The 3 most-used cell types and their counts	`dfrtp_1`: 16, `nand2_1`: 7, `and2_0` and `maj3_1`: 4 each
Netlist simulation result (PASS/FAIL line)	`PASS: 256 vectors checked, 0 errors`

4. Part C: Static Timing Analysis with OpenSTA

`sta/sta.tcl` runs the seven OpenSTA steps: it reads the typical-corner library and the Part B netlist, links `mult_array`, creates the clock `clk` with period T, sets 1 ns input and output delays on all other ports, prints the worst setup slack and writes the worst path of each path group to `sta/mult_array_timing.rpt`. The clock and the wires are ideal, because nothing has been placed or routed yet.

At T = 10 ns the worst setup slack is WNS = 7.213 ns, so T_min = T − WNS = 10 − 7.213 = 2.787 ns and F_max = 1000 / 2.787 = 358.8 MHz. Running the script again with T_min rounded to 0.01 ns, T = 2.79 ns, gives a worst slack of +0.003 ns (MET), close to 0 as expected. The committed `sta/sta.tcl` and `sta/mult_array_timing.rpt` are for this run.

The critical path (Table 5) runs from the input register of a[1] (`_91_`) to the output register of p[6] (`_88_`): 0.398 ns clock-to-Q, 2.254 ns through 8 logic cells and 0.135 ns setup time, 2.787 ns in total, which is T_min. Because the inputs and outputs are registered, the 1 ns input and output delays only apply to the short paths at the ports, and the reset path (a recovery check on `rst_n`, 9.228 ns slack at T = 10 ns) is not critical either.

Table 4. Static timing results (typical corner tt_025C_1v80).

Metric	Value
Worst slack at T = 10 ns (ns)	7.213
T_min (ns)	2.787 (= 10 − 7.213)
F_max (MHz)	358.8 (= 1000 / 2.787)
Slack at T = T_min (ns)	+0.003 (MET), at T = 2.79 ns (T_min rounded to 0.01 ns)
Critical path start point	`_91_` (`dfrtp_1`), input register a_r[1]
Critical path end point	`_88_` (`dfrtp_1`), output register p[6]
Number of cells on the critical path	8 logic cells (plus 2 flip-flops)
Latency (ns) = 2 × T_min	5.574
Throughput (million results per second)	358.8 (one product per cycle)

Table 5. Cells on the critical path, in order (cell names without the sky130_fd_sc_hd__ prefix).

#	Instance	Cell type	Incr. (ns)	Arrival (ns)	Origin
0	`_91_`	`dfrtp_1`	0.398	0.398	Launch flip-flop a_r[1] (clock to Q)
1	`_37_`	`clkinv_1`	0.073	0.471	Inverts a_r[1]
2	`_42_`	`o311ai_0`	0.356	0.827	Row 1, bit 1: part of sum s1[1] (a1·b1, a2·b0 merged in)
3	`_45_`	`a21oi_1`	0.153	0.980	Ends s1[1]; row 2, bit 0 carry c2[1]
4	`_51_`	`maj3_1`	0.402	1.382	Row 2, bit 1 carry c2[2]
5	`_58_`	`xnor3_1`	0.370	1.752	Row 2, bit 2 sum s2[2] (inverted)
6	`_61_`	`maj3_1`	0.368	2.119	Row 3, bit 1 carry c3[2] (inverted)
7	`_63_`	`maj3_1`	0.354	2.474	Row 3, bit 2 carry c3[3] (inverted)
8	`_64_`	`xor3_1`	0.178	2.652	Row 3, bit 3 sum s3[3] = p_next[6]
–	`_88_`	`dfrtp_1`	0.135 (setup)	2.787 (with setup)	Capture flip-flop p[6]

5. Part D: Physical Design with LibreLane

`pnr/config.json` gives LibreLane the two RTL files, the clock port `clk`, the clock period and a fixed 150 µm × 150 µm die (`FP_SIZING` = `absolute`). The first `CLOCK_PERIOD` was 1.2 × T_min = 1.2 × 2.787 = 3.34 ns, rounded up to the next 0.5 ns, so 3.5 ns. LibreLane synthesises the RTL again with its own Yosys script (62 cells, 758.227 µm²) and then runs floorplanning, the power grid,

placement, clock tree synthesis, timing repair, routing, parasitic extraction, STA, GDS stream-out, DRC and LVS. In this container it checks setup timing at all nine corners (typical, slow and fast transistors, each with minimum, nominal and maximum wire parasitics) and stops with an error if any of them has a violation.

Every run up to 7.5 ns stopped with setup violations at the three slow corners (`ss_100C_1v60`: slow transistors, 100 °C, 1.60 V), although the typical corner had positive slack in every run. The period was increased by 0.5 ns after each failure, and the flow completed at 8.0 ns in the 10th run (Table 6). Up to 5.5 ns the timing repair after clock tree synthesis could not remove the violations (warning RSZ-0062). From 6.0 ns on it no longer saw any, but it works with wire parasitics estimated from the placement, and after routing the extracted parasitics left the slow-corner slack at −0.10 to −0.22 ns. The 7.5 ns and 8.0 ns layouts are practically identical (same cells, same 2499 µm of wire), so the extra 0.5 ns turned −0.104 ns into +0.396 ns.

Table 6. All LibreLane runs, in the order they were made: worst setup slack over all corners (timing__setup__ws, always at max_ss_100C_1v60) and at the typical corner.

Run	Clock period (ns)	Worst setup slack (ns)	Setup slack at nom_tt_025C_1v80 (ns)	Result
1	3.5	−2.188	+0.509	fail
2	4.0	−1.688	+1.009	fail
3	4.5	−1.188	+1.509	fail
4	5.0	−0.686	+2.009	fail
5	5.5	−0.208	+2.509	fail
6	6.0	−0.154	+2.877	fail
7	6.5	−0.220	+3.119	fail
8	7.0	−0.134	+3.520	fail
9	7.5	−0.104	+3.737	fail
10	8.0	+0.396	+4.237	pass

The final run (`runs/RUN_2026-10-26_18-52-09`, Table 7) has 0 DRC errors (Magic and KLayout), 0 LVS errors, no antenna violations and a worst hold slack of +0.136 ns. Its post-route F_max is 1000 / (8.0 − 0.396) = 131.5 MHz at the worst corner and 1000 / (8.0 − 4.237) = 265.8 MHz at the typical corner, against 358.8 MHz in Part C. The flow also reports 11 max-slew warnings at the slow corners, all on the reset net driven by the fanout buffer `fanout20` (0.821 ns against a 0.75 ns limit); they do not stop the flow, and the recovery checks on these pins pass with more than 5 ns of slack. Of the 1883 cells, 62 are the unchanged logic cells from synthesis; the flow added 1797 tap, decap and fill cells and 24 buffers (Table 8). Figures 4 and 5 show the layout.

Table 7. Physical design results.

Metric	Value
CLOCK_PERIOD used (ns)	8.0
Number of runs needed	10 (3.5 to 8.0 ns, 0.5 ns steps)
Core area (µm²)	17759.5 (die 22500)
Cells after synthesis (from Part B)	61 (62 in LibreLane's own synthesis)
Cells after place and route	1883 (62 logic, 24 buffers, 1797 tap/decap/fill)
Kinds of cells added	Tap, decap, fill; clock, port, fanout and wire buffers (Table 8)
Wirelength (µm)	2499 (detailed routing)
Setup slack (ns)	+0.396 (max_ss_100C_1v60); +4.237 (nom_tt)
Post-route F_max (MHz)	131.5 (worst corner); 265.8 (typical)
F_max from Part C (MHz)	358.8
DRC errors	0 (Magic and KLayout)
LVS errors	0 (Netgen)

Table 8. Cell types added by place and route.

Cell type	Count	Purpose
`decap_3/4/6/8/12`	1298	Decoupling capacitors between VPWR and VGND: keep the supply steady when cells switch.
`fill_1`, `fill_2`	254	Fill the gaps in the rows so wells, implants and power rails are continuous.
`tapvpwrvgnd_1`	245	Well taps: tie the n-well to VPWR and the substrate to VGND; prevent latch-up.
`buf_2`	17	Port buffers on the 9 inputs and 8 outputs: isolate the logic from outside loads.
`clkbuf_16`	3	Clock tree: one root and two leaf buffers (8 flip-flops each), small skew.
`clkdlybuf4s50_1`	2	Fanout buffers (`fanout20`, `fanout21`) on the reset net (16 flip-flops).
`clkdlybuf4s25_1`	2	Wire repeater (`wire19`) and max-cap buffer (`max_cap18`) on a long critical-path net.

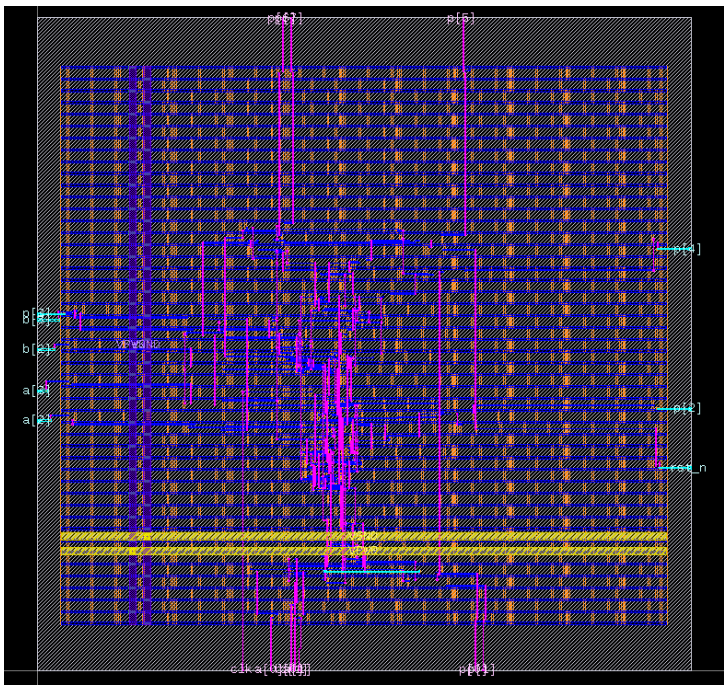


Figure 4. Final layout of the whole 150 µm × 150 µm die in KLayout (pnr/mult_array.gds). The blue horizontal lines are the met1 power rails of the standard-cell rows, and the purple vertical and yellow horizontal straps are the met4 and met5 power grid. The logic cells and their routing (pink met2, blue met1) sit in the centre of the die; the rest of the rows are filled with decap, fill and tap cells. The pins are on the die edges.

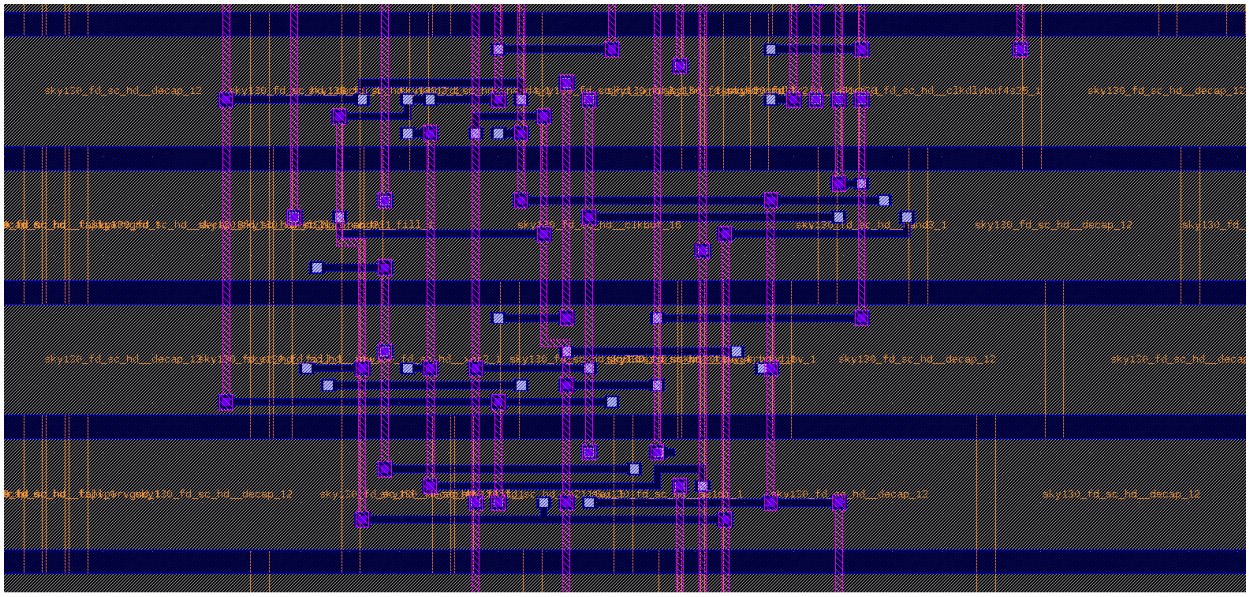


Figure 5. Zoomed view of the centre of the die: standard-cell rows between the wide met1 power rails, with the cell outlines and names (for example decap_12, clkbuf_16, and3_1), the met1 (blue, horizontal) and met2 (pink, vertical) signal wires and the vias between them (squares).

6. Answers to the Questions

Q1. The array needs 16 AND gates and 12 full adders. How many cells did synthesis produce for the logic (total cells minus flip-flops)? Explain why the number of cells is not simply 16 + 12.

Synthesis produced 61 cells, of which 16 are flip-flops, so the logic needs 61 − 16 = 45 cells instead of 16 + 12 = 28. The 28 blocks of the RTL do not survive as such: `synth -flatten` removes the 12 `full_adder` modules, and ABC maps the whole combinational network onto single-output gates only, skipping the library's multi-output full-adder cell `fa_1` (the log warns that it

detected 9 multi-output cells), so each adder becomes separate sum logic (`xor2_1`, `xor3_1`, `xnor2_1`, `xnor3_1`) and carry logic (`maj3_1` or AND-OR-INVERT gates). Most of the 16 AND gates are merged into compound gates such as `a21oi_1`, `a22oi_1` and `o2111ai_1` (only 4 `and2_0`, one `and3_1` and one `and4_1` remain), the adders with a constant 0 input shrink to half adders, and inverting gates plus 2 `clkinv_1` are used where they are cheaper. The count therefore reflects ABC's optimisation of the whole network, not the 28 blocks written in the RTL.

Q2. What is the critical path of your design, from which register to which register? How many cells are on it, and how does this compare with the estimate of about 2N full-adder delays?

The critical path runs from the input register a_r[1] (flip-flop `_91_`) to the output register p[6] (flip-flop `_88_`) and has 8 logic cells between the two flip-flops (Table 5). Traced back to the RTL, it goes diagonally down the array and then along the carry chain of the last row: row 1 bit 1 → row 2 bits 0, 1 and 2 → row 3 bits 1, 2 and 3 (p[6]), that is 7 adder positions plus the AND-gate level, about 8 stages, which agrees with the estimate of about 2N = 8 full-adder delays. Of the 8 cells, 5 are recognisable full-adder cells (the three `maj3_1` carries and the `xnor3_1` and `xor3_1` sums), 2 (`o311ai_0` and `a21oi_1`) combine partial-product AND gates with the logic of the first adders, and 1 is an inverter. The 8 cells take 2.254 ns, about 0.28 ns per stage.

Q3. What percentage of the total cell area is flip-flops? Why is this percentage large for such a small design?

The 16 flip-flops take 400.384 µm² of the 756.976 µm² total, which is 52.9 %. Each `dfrtp_1` (a D flip-flop with asynchronous reset: a master and a slave latch, clock buffering and reset transistors) is 25.024 µm², about 6.7 times a `nand2_1` (3.754 µm²), while the average logic cell here is only 7.9 µm² (356.592 µm² over 45 cells). The number of flip-flops is fixed by the interface (4 + 4 input bits and 8 output bits), whereas the logic of a 4-bit multiplier is still small. For an N-bit multiplier the logic grows roughly as N² but the registers only as 4N, so the flip-flop share would fall for wider designs.

Q4. Why is the post-route F_max lower than the F_max from Part C? Which kinds of cells did the flow add, and why is each kind needed?

The post-route F_max is 131.5 MHz at the worst corner (265.8 MHz at the typical corner), against 358.8 MHz in Part C, which assumed one typical corner, an ideal clock and no wires. After place and route the flow signs off at the slow corner `max_ss_100C_1v60`, which roughly doubles every cell delay (clock-to-Q of the launching flip-flop 0.992 ns against 0.515 ns at the typical corner, each `maj3_1` about 0.92 ns against 0.42 ns); the routed wires add resistance and capacitance, and two buffers on a long net of the critical path (`wire19`, `max_cap18`) add 0.61 ns at the typical corner and 1.23 ns at the slow corner; and the clock tree adds skew, with 0.25 ns of clock uncertainty on top. The flow added 1821 cells (Table 8): tap cells, which tie the n-well and substrate to the supplies and prevent latch-up; decap cells, which keep the supply steady when cells switch; fill cells, which keep the wells and power rails continuous; clock-tree buffers, so that all 16 flip-flops see the clock edge at nearly the same time; port buffers, which isolate the logic from external drivers and loads; and fanout and wire buffers, which keep slew and capacitance within the library limits.

Q5. What does a DRC check verify, and what does an LVS check verify? Why must both pass before a chip can be manufactured?

DRC (design rule check) verifies that the layout geometry obeys the manufacturing rules of the process: minimum width, spacing, enclosure, area and density of the shapes on every layer. A violation means the foundry cannot reliably make the shape, so a wire could break or two wires could short. LVS (layout versus schematic) extracts the transistors and their connections from the layout and compares them with the netlist, so it verifies that the layout is exactly the intended circuit: no missing or extra devices, no open or shorted nets and the correct pins. Both must pass because they catch different errors: a DRC-clean layout can still be the wrong circuit, an LVS-clean

layout can still be impossible to manufacture, and a mask set is far too expensive to find either on silicon. The final run passed both: 0 DRC errors in Magic and KLayout and 0 LVS differences in Netgen.

Q6. Using the formulas in Section 2 of the assignment, estimate how the number of full adders and the critical path delay would change if the multiplier were 8 bits wide instead of 4. Show your working.

With the formulas of Section 2, an N-bit array multiplier has N^2 AND gates, $N(N-1)$ full adders and a critical path of about 2N full-adder delays. For N = 4 this gives 16 AND gates, 4 × 3 = 12 full adders and about 8 delays; for N = 8 it gives 64 AND gates, 8 × 7 = 56 full adders and about 16 delays. The number of full adders therefore grows 56 / 12 ≈ 4.7 times (roughly as N^2), while the critical path only doubles (linearly with N). In time, Part C measured 2.254 ns of logic for about 8 full-adder delays, about 0.28 ns each, so the 8-bit logic delay would be about 16 × 0.28 = 4.51 ns. With the same flip-flop overhead (0.398 ns clock-to-Q and 0.135 ns setup), T_min ≈ 0.398 + 4.51 + 0.135 ≈ 5.04 ns and F_max ≈ 1000 / 5.04 ≈ 198 MHz, against 2.787 ns and 358.8 MHz for the 4-bit design.

7. Conclusion

A registered 4-bit unsigned array multiplier was taken from RTL to a DRC- and LVS-clean GDSII layout on SKY130 with open-source tools only. The RTL passed an exhaustive self-checking simulation of all 256 input pairs, and a deliberately broken full adder showed that the testbench detects errors. Yosys mapped the design onto 61 cells (756.976 µm², 52.9 % of it flip-flops), and the gate-level netlist passed the same 256 vectors.

At the typical corner, with an ideal clock and no wires, OpenSTA gave T_min = 2.787 ns (358.8 MHz), with an 8-cell critical path from a_r[1] to p[6] that agrees with the estimate of about 2N = 8 full-adder delays. LibreLane, which here requires all nine corners to meet setup timing, completed at CLOCK_PERIOD = 8.0 ns after 10 runs; the post-route F_max is 131.5 MHz at the slow corner and 265.8 MHz at the typical corner. The difference comes from the slow corner, the real wires and the buffers inserted on the critical path, and the clock tree with its uncertainty margin. The 62 logic cells, 24 buffers and the taps cover only 7.1 % of the core; the rest of the rows is decap and fill.

Appendix A: Cell types after synthesis (synth/mult_array_stat.rpt, without the sky130_fd_sc_hd__ prefix)

Cell type	Count	Area (µm²)	Cell type	Count	Area (µm²)
a21boi_0	1	7.507	nand3_1	2	10.01
a21oi_1	3	15.014	nand4_1	2	12.512
a22oi_1	2	15.014	nor2_1	2	7.507
and2_0	4	25.024	o2111ai_1	1	8.758
and3_1	1	6.256	o21bai_1	1	7.507
and4_1	1	8.758	o311ai_0	1	8.758
clkinv_1	2	7.507	xnor2_1	3	26.275
dfrtp_1	16	400.384	xnor3_1	1	22.522
lpflow_isobufsrc_1	2	12.512	xor2_1	2	17.517
maj3_1	4	40.038	xor3_1	3	71.318
nand2_1	7	26.275	Total	61	756.976